MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 32 — Structures de Données Avancées

collections, heapq, bisect — les structures que les experts utilisent

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

30 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 32 — Structures de Données Avancées **2**
0.1 defaultdict — Dictionnaire avec valeur par défaut 2
0.2 Counter — Compter les occurrences 3
0.3 deque — File double à deux bouts 4
0.4 heapq — File de priorité 6
0.5 bisect — Recherche et insertion binaire 7
0.6 OrderedDict et ChainMap 8
0.7 Ce qui m'a le plus surpris dans ce chapitre 9
0.8 Résumé du Chapitre 32 10

Chapitre 32 — Structures de Données Avancées

À retenir

Au-delà des listes, tuples, dicts et sets, Python offre un arsenal de structures de données spécialisées dans le module `collections` et les modules `heapq` et `bisect`. Chacune est conçue pour exceller dans un cas d'usage précis : comptage, files d'attente, accès ordonné, recherche binaire. Les connaître et les utiliser au bon moment est ce qui distingue le code d'un expert du code d'un débutant.

defaultdict — Dictionnaire avec valeur par défaut

collections.defaultdict

```python
from collections import defaultdict

# Le probleme avec dict normal
compteur_normal = {}
mots = ["python","est","g nial","python","est","python"]
for mot in mots:
    if mot not in compteur_normal:   # verification requise
        compteur_normal[mot] = 0
    compteur_normal[mot] += 1
print(compteur_normal)

# defaultdict     plus elegant
compteur = defaultdict(int)   # int() retourne 0
for mot in mots:
    compteur[mot] += 1        # pas de verification !
print(dict(compteur))

# defaultdict(list)     grouper des elements
etudiants = [
    ("Paris",    "Alice"),
    ("Lyon",     "Bob"),
    ("Paris",    "Charlie"),
    ("Marseille","Diana"),
    ("Lyon",     "Eve"),
]

par_ville = defaultdict(list)
for ville, prenom in etudiants:
    par_ville[ville].append(prenom)

for ville, prenoms in sorted(par_ville.items()):
    print(f"  {ville:12} : {', '.join(prenoms)}")
```

```python

# defaultdict(set)      valeurs uniques par cle
tags_articles = defaultdict(set)
donnees = [
    ("python",     "programmation"),
    ("python",     "data-science"),
    ("python",     "programmation"),    # doublon ignore
    ("javascript","web"),
    ("python",     "web"),
]
for langage, tag in donnees:
    tags_articles[langage].add(tag)

for lang, tags in tags_articles.items():
    print(f"  {lang}: {sorted(tags)}")

# defaultdict imbrique
graphe = defaultdict(lambda: defaultdict(int))
aretes = [("A","B",5), ("A","C",3), ("B","C",2)]
for src, dst, poids in aretes:
    graphe[src][dst] = poids
    graphe[dst][src] = poids    # non-oriente

print(dict(graphe["A"]))    # {'B': 5, 'C': 3}
```

Counter — Compter les occurrences

collections.Counter — comptage intelligent

```python
from collections import Counter

# Creer un Counter
mots = ["python","est","g nial","python","est","python",
        "data","science","python","data"]
c = Counter(mots)
print(c)
# Counter({'python': 4, 'est': 2, 'data': 2, ...})

# Depuis une chaine
lettres = Counter("mississippi")
print(lettres)
# Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})

# Operations arithmetiques sur Counter
c1 = Counter(["a","b","a","c","a","b"])
c2 = Counter(["a","b","b","d"])

print(c1 + c2)    # addition
```

```python
print(c1 - c2)    # soustraction (garde positifs)
print(c1 & c2)    # intersection (min)
print(c1 | c2)    # union (max)

# most_common()      les N plus frequents
texte = """Python est un langage de programmation
           Python est populaire en data science
           Python est utilise partout"""
mots  = texte.lower().split()
freq  = Counter(mots)

print("\nTop 5 mots :")
for mot, n in freq.most_common(5):
    barre = "    " * n
    print(f"  {mot:15} {barre} ({n})")

# Cas pratique      analyse fr quentielle
notes = [15, 18, 12, 15, 18, 18, 12, 9, 15, 17]
dist  = Counter(notes)
print("\nDistribution des notes :")
for note in sorted(dist.keys(), reverse=True):
    barre = "    " * dist[note]
    print(f"  {note:2}/20  {barre}  ({dist[note]})")

# update()      ajouter des donnees
dist.update([15, 15, 18])
print(f"\n Apres update : {dict(dist)}")

# elements()      iterer sur les elements
print(list(Counter({"a":3,"b":2}).elements()))
# ['a', 'a', 'a', 'b', 'b']
```

deque — File double à deux bouts

collections.deque — O(1) aux deux extrémités

```python
from collections import deque

# deque      Double Ended Queue
# O(1) pour append/pop des DEUX cotes
# O(n) pour list (insert/pop en debut de liste)

# Creation
d = deque([1, 2, 3, 4, 5])
print(d)    # deque([1, 2, 3, 4, 5])

# Ajouter aux deux extremit s
d.append(6)         # ajouter a droite
```

```python
d.appendleft(0)     # ajouter a gauche
print(d)    # deque([0, 1, 2, 3, 4, 5, 6])

# Supprimer aux deux extremites
droite = d.pop()        # supprimer a droite
gauche = d.popleft()    # supprimer a gauche
print(f"Pop droite : {droite}, gauche : {gauche}")
print(d)

# Ajouter plusieurs elements
d.extend([7, 8, 9])
d.extendleft([-2, -1])  # attention     ordre inverse !
print(d)

# Rotation
d = deque([1, 2, 3, 4, 5])
d.rotate(2)     # d caler a droite
print(d)    # deque([4, 5, 1, 2, 3])
d.rotate(-2)    # d caler a gauche
print(d)    # deque([1, 2, 3, 4, 5])

# maxlen     file circulaire bornee
historique = deque(maxlen=5)
for i in range(10):
    historique.append(f"cmd_{i}")
print(historique)
# deque(['cmd_5','cmd_6','cmd_7','cmd_8','cmd_9'], maxlen=5)

# Cas pratique     file de taches (BFS)
def bfs(graphe: dict, depart: str) -> list:
    """Parcours en largeur avec deque."""
    visites   = []
    a_visiter = deque([depart])
    vus       = {depart}

    while a_visiter:
        sommet = a_visiter.popleft()
        visites.append(sommet)
        for voisin in graphe.get(sommet, []):
            if voisin not in vus:
                vus.add(voisin)
                a_visiter.append(voisin)
    return visites

graphe = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"], "E": ["B"], "F": ["C"]
}
```

```
print(bfs(graphe, "A"))
# ['A', 'B', 'C', 'D', 'E', 'F']
```

heapq — File de priorité

heapq — tas min en Python

```
import heapq

# Python implemente un MIN-heap
# L'   lment    le plus PETIT est toujours en tete

nombres = [3, 1, 4, 1, 5, 9, 2, 6]
heapq.heapify(nombres)    # transformer en tas en place
print(nombres)    # [1, 1, 2, 6, 5, 9, 4, 3]

# Pousser et extraire
heapq.heappush(nombres, 0)
print(heapq.heappop(nombres))    # 0 (le plus petit)
print(heapq.heappop(nombres))    # 1

# Voir le minimum sans extraire
print(nombres[0])    # toujours le minimum

# nsmallest et nlargest
notes = [15, 18, 12, 9, 17, 14, 11, 16, 8, 19]
print(heapq.nsmallest(3, notes))    # [8, 9, 11]
print(heapq.nlargest(3, notes))     # [19, 18, 17]

# File de priorite      (priorite, donnees)
taches = []
heapq.heappush(taches, (3, "Tache faible priorite"))
heapq.heappush(taches, (1, "Tache urgente"))
heapq.heappush(taches, (2, "Tache normale"))
heapq.heappush(taches, (1, "Aussi urgente"))

print("\nTraitement par priorite :")
while taches:
    prio, tache = heapq.heappop(taches)
    print(f"  [P{prio}] {tache}")

# MAX-heap      inverser le signe
max_heap = []
for n in [3, 1, 4, 1, 5, 9]:
    heapq.heappush(max_heap, -n)

print("\n Max heap (ordre decroissant) :")
while max_heap:
```

```
    print(-heapq.heappop(max_heap), end=" ")
print()

# Fusion de listes triees      O(n log k)
listes = [[1,4,7], [2,5,8], [3,6,9]]
print("\nFusion triee :")
print(list(heapq.merge(*listes)))
```

bisect — Recherche et insertion binaire

bisect — maintenir une liste triée

```
import bisect

# bisect trouve la position d'insertion pour garder le tri
notes_triees = [9, 12, 14, 15, 17, 18]

# bisect_left      position avant les egaux
pos = bisect.bisect_left(notes_triees, 15)
print(f"Position de 15 (left)  : {pos}")   # 3

# bisect_right      position apres les egaux
pos = bisect.bisect_right(notes_triees, 15)
print(f"Position de 15 (right) : {pos}")   # 4

# insort      inserer en maintenant l'ordre trie
bisect.insort(notes_triees, 16)
print(notes_triees)   # [9, 12, 14, 15, 16, 17, 18]

bisect.insort_left(notes_triees, 15)
print(notes_triees)   # [9, 12, 14, 15, 15, 16, 17, 18]

# Cas pratique      trouver la mention
def get_mention(note: float) -> str:
    """Recherche binaire O(log n) vs if-elif O(n)."""
    seuils   = [10, 12, 14, 16]
    mentions = ["Echec", "Passable", "Assez Bien",
                "Bien",  "Tres Bien"]
    return mentions[bisect.bisect(seuils, note)]

notes_test = [9.5, 10.0, 13.5, 15.0, 17.5, 20.0]
for n in notes_test:
    print(f"  {n:5.1f}/20     {get_mention(n)}")

# Recherche dans une liste triee      O(log n)
def recherche_binaire(liste: list, cible) -> int:
    """Retourne l'index ou -1 si absent."""
    pos = bisect.bisect_left(liste, cible)
```

```python
    if pos < len(liste) and liste[pos] == cible:
        return pos
    return -1

data = sorted([15, 18, 12, 9, 17, 14, 11, 16, 8, 19])
print(f"\nRecherche 17 : index {recherche_binaire(data, 17)}")
print(f"Recherche 13 : index {recherche_binaire(data, 13)}")
```

OrderedDict et ChainMap

OrderedDict et ChainMap

```python
from collections import OrderedDict, ChainMap

# OrderedDict      dict avec ordre garanti (Python 3.7+: dict
   normal)
# Utile pour : comportements speciaux comme move_to_end
od = OrderedDict([("a",1), ("b",2), ("c",3)])

# Deplacer un element en fin ou en debut
od.move_to_end("a")          # deplacer en fin
print(list(od.keys()))       # ['b', 'c', 'a']

od.move_to_end("a", last=False)  # deplacer en debut
print(list(od.keys()))       # ['a', 'b', 'c']

# popitem() depuis la fin ou le debut
last  = od.popitem(last=True)    # ('c', 3)
first = od.popitem(last=False)   # ('a', 1)
print(last, first)

# LRU Cache manuel avec OrderedDict
class LRUCache:
    def __init__(self, capacite: int):
        self.cache     = OrderedDict()
        self.capacite  = capacite

    def get(self, cle):
        if cle not in self.cache:
            return -1
        self.cache.move_to_end(cle)    # recemment utilise
        return self.cache[cle]

    def put(self, cle, valeur):
        if cle in self.cache:
            self.cache.move_to_end(cle)
        self.cache[cle] = valeur
        if len(self.cache) > self.capacite:
```

```
            self.cache.popitem(last=False)   # expulser LRU

lru = LRUCache(3)
lru.put("a", 1); lru.put("b", 2); lru.put("c", 3)
print(lru.get("a"))     # 1      "a" devient le plus recent
lru.put("d", 4)         # "b" est expuls  (LRU)
print(lru.get("b"))     # -1 (expulse)

# ChainMap      vue unifiee de plusieurs dicts
defauts  = {"debug": False, "port": 8000, "host": "localhost"}
env_vars = {"port": 9000, "db": "production"}
args_cli = {"debug": True}

# Priorite : args_cli > env_vars > defauts
config = ChainMap(args_cli, env_vars, defauts)

print(f"\n Configuration finale :")
print(f"  debug : {config['debug']}")    # True (args_cli)
print(f"  port  : {config['port']}")     # 9000 (env_vars)
print(f"  host  : {config['host']}")     # localhost (defauts)
print(f"  db    : {config['db']}")       # production (env_vars)

# Ajouter une couche
config_locale = config.new_child({"port": 7777})
print(f"  port (local) : {config_locale['port']}")  # 7777
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `Counter` fait en une ligne ce qui prenait dix. Compter les occurrences, trouver les plus fréquents, combiner des compteurs — tout ce que j'implémentais avec des boucles `defaultdict(int)` et des conditions. `Counter` encapsule tout ça avec une API élégante et des opérations arithmétiques directes. C'est l'exemple parfait de Python qui offre la solution au problème exact que tout le monde rencontre.

2. `deque` vs `list` pour les files — une différence de complexité. Insérer en début de liste avec `list.insert(0, x)` déplace tous les éléments — O(n). `deque.appendleft(x)` est O(1). Pour une file de 100 000 éléments, la différence est spectaculaire. Cette leçon m'a rappelé que chaque structure de données a ses opérations optimales — et qu'utiliser la mauvaise structure au mauvais endroit est une source silencieuse de lenteur.

3. `bisect` pour les mentions — O(log n) vs O(n). J'avais une chaîne de `if/elif` pour calculer les mentions. `bisect.bisect(seuils, note)` fait exactement la même chose en une ligne, avec une complexité logarithmique plutôt que linéaire. Pour un tableau de correspondance fixe, la recherche binaire est toujours plus propre et plus

rapide que les conditions imbriquées. Une révélation pour mes projets d'analyse statistique.

Résumé du Chapitre 32

Ce que j'ai appris dans ce chapitre :

- `defaultdict` — valeur par défaut automatique, groupement élégant.
- `Counter` — comptage, `most_common()`, opérations arithmétiques.
- `deque` — O(1) aux deux extrémités, `maxlen` pour fenêtres glissantes.
- `heapq` — tas min, file de priorité, `nsmallest`/`nlargest`.
- `bisect` — recherche et insertion binaire O(log n).
- `OrderedDict` — `move_to_end()`, LRU cache manuel.
- `ChainMap` — vue prioritaire sur plusieurs dictionnaires.
- Chaque structure a ses opérations O(1) — choisir selon le cas d'usage.

"La structure de données appropriée transforme un algorithme $O(n^2)$ en $O(n \log n)$. Connaître son arsenal, c'est penser en complexité avant de coder."